

The Purple Cubby

Multi-Tenant CRM for Educational Institutions
System Architecture & Technical Reference

Version 2.0 — Consolidated	Status Architecture Complete
Institutions Childcare · Pre-school · Montessori · Private · Government	Coverage 10 Portals · 23 BOs · 46 Entities · 10 Processes

1. System Architecture

1.1 Overview

The Purple Cubby is a cloud-native, multi-tenant CRM platform serving educational institutions from home-based childcare to government school networks. Each subscribing institution operates in a fully isolated tenant environment with custom branding, while sharing the underlying platform infrastructure.

1.2 Three-Layer Architecture

The system is organised into three strict layers. No layer may communicate with a layer it is not adjacent to. The presentation layer never touches base entities directly.

Layer	Components	Responsibility
Presentation	10 role-based portals, REST/GraphQL API, Mobile apps, Webhooks	User interaction, API contracts, portal routing — reads/writes only through BOs
Business Object	23 Business Objects across 6 domains	Computed properties, business rules enforcement, cross-entity assembly, state machine management
Data Entity	46 normalised database tables, audit log, file storage	Persistent storage, referential integrity, row-level tenant security, audit trail

1.3 Multi-Tenancy

Every database table carries a tenant_id column. Row-level security (RLS) policies at the database layer enforce that all queries are automatically scoped to the authenticated tenant. No application-layer filtering is relied upon for tenant isolation — RLS is the enforcement boundary.

1.4 Subscription Tiers

Module	Basic	Standard	Enterprise
Enrollment & Admissions	✓	✓	✓
Billing & Payments	✓	✓	✓
Attendance Tracking	✓	✓	✓
Care Giver Module	✓	✓	✓
Health & Safety	✓	✓	✓

Academic Management	—	✓	✓
HR & Payroll	—	✓	✓
Vendor Management	—	✓	✓
Events & Extra-Curricular	—	✓	✓
Multi-Branch / Network Analytics	—	—	✓
Custom Report Builder	—	—	✓
API & Webhook Access	—	—	✓

2. Portals Supported

The Purple Cubby provides ten distinct role-based portals. Each portal exposes only the modules, data, and actions relevant to that role. All portals share the same underlying API — access is enforced by the RBAC engine, not by separate codebases.

Role	Tier	Portal Access & Capabilities
Owner	Network	Full cross-school analytics, subscription management, payroll approval, fee policy, emergency broadcast, spend analytics, vendor contracts
Board Member	Governance	Financial dashboards, compliance reports, audit trails, vendor contract approval, enrollment KPIs — read-only
Trustee	Governance	Fund utilisation, trust compliance, charitable reporting, board-level financial view — read-only
Principal	School	Full school operations: staff, academics, events, communications, admissions decisions, safety compliance
Vice Principal	School	Staff scheduling, substitution, lesson plan review, exam timetabling, day-to-day operations
Administrator	School	Billing, enrollment processing, HR records, vendor management, report generation, system configuration
Teacher	Classroom	Attendance, lesson plans, assignments, marks entry, class messaging, leave requests
Care Giver	Classroom	Care session attendance, health observations, medication logs, daily parent updates
Parent	Self-Service	Child progress, fee payment, attendance alerts, event registration, permission slips, direct messaging
Vendor	External	Invoice submission, PO tracking, delivery confirmation, payment status, account profile

All portals are responsive web applications with PWA support for mobile. The Teacher and Care Giver portals have offline-capable attendance marking that syncs when connectivity is restored.

3. Entities and Business Objects

3.1 The 46 Base Entities

All 46 entities are normalised to 3NF. Every entity carries `tenant_id`, `created_at`, `updated_at`, and `created_by` as standard audit columns in addition to the fields listed below.

Entity	Domain	Key Fields
TENANT	Root	Tenant ID, name, subdomain, type, tier, region, config
STUDENT	Student	Student ID, tenant FK, class FK, name, dob, grade, status, enrolled date
APPLICATION	Student	Application ID, student FK, status, documents, decision, decided_by
PARENT	Student	Parent ID, student FK, name, relationship, phone, email, notification_prefs
EMERGENCY_CONTACT	Student	Contact ID, student FK, name, phone, relationship, consent
DOCUMENT	Student	Doc ID, application FK, type, file_path, verified flag
STUDENT_HEALTH	Health	Health ID, student FK, allergies, conditions, medications, vaccinations
HEALTH_OBSERVATION	Health	Obs ID, student FK, staff FK, date, mood, appetite, nap, feeding
MEDICATION_LOG	Health	Log ID, student FK, staff FK, medicine, dose, time_administered
INCIDENT	Health	Incident ID, tenant FK, type, date, location, involved parties, actions
SAFETY_DRILL	Health	Drill ID, tenant FK, type, date, duration, participation_rate
CURRICULUM	Academic	Curriculum ID, grade, subjects, outcomes, version, approved_by
ACADEMIC_CALENDAR	Academic	Calendar ID, tenant FK, year, terms, holidays, exam_weeks
LESSON_PLAN	Academic	Plan ID, teacher FK, class FK, curriculum FK, week, topic, status
ASSIGNMENT	Academic	Assignment ID, teacher FK, class FK, title, due_date, submissions
EXAM	Academic	Exam ID, class FK, calendar FK, name, type, date, room, invigilator
MARKS_RECORD	Academic	Marks ID, student FK, exam FK, subject, marks, grade, locked
REPORT_CARD	Academic	Report ID, student FK, term, year, overall_grade, published_date
STUDENT_ATTENDANCE	Attendance	Att ID, student FK, class FK, date, status, method, marked_by

STAFF_ATTENDANCE	Attendance	Staff Att ID, staff FK, date, check_in, check_out, method, status
LEAVE_REQUEST	Attendance	Leave ID, requester FK, type, dates, days, status, approved_by
ATTENDANCE_ROSTER	Attendance	Roster ID, class FK, tenant FK, date, window_start, window_end
FEE_STRUCTURE	Billing	Fee Struct ID, tenant FK, grade, term, components, rules, version
INVOICE	Billing	Invoice ID, student FK, parent FK, fee_struct FK, line_items, total, status
PAYMENT	Billing	Payment ID, invoice FK, parent FK, amount, method, txn_ref, status
DISCOUNT_WAIVER	Billing	Discount ID, invoice FK, type, amount, reason, approved_by
RECONCILIATION	Billing	Recon ID, tenant FK, period, total_invoiced, collected, outstanding
STAFF	HR	Staff ID, tenant FK, name, role, dept, employment_type, salary, certifications
CONTRACT	HR	Contract ID, staff FK, type, start, end, esigned_at, renewal_alert
PAYROLL_RUN	HR	Payroll ID, staff FK, period, components, gross, net, status, approved_by
APPRAISAL	HR	Appraisal ID, staff FK, appraiser FK, cycle, scores, outcome, status
CPD_RECORD	HR	CPD ID, staff FK, activity, type, hours, mandatory, completion
SCHEDULE	HR	Schedule ID, staff FK, class FK, week, slot, duty_type, substitute_flag
VENDOR	Vendor	Vendor ID, tenant FK, name, tax_reg, bank_details, category, status, score
REQUISITION	Vendor	Req ID, raised_by, items, total, approval_tier, status, approved_by
PURCHASE_ORDER	Vendor	PO ID, req FK, vendor FK, items, delivery_date, terms, status
DELIVERY_RECORD	Vendor	Delivery ID, PO FK, items_received, discrepancies, confirmed_by
VENDOR_INVOICE	Vendor	V-Invoice ID, PO FK, vendor FK, total, match_result, approval_status
INVENTORY	Vendor	Item ID, tenant FK, name, stock_level, min_threshold, location
EVENT	Events	Event ID, tenant FK, name, type, date, venue, capacity, budget, status
EVENT_REGISTRATION	Events	Reg ID, event FK, student FK, status, permission_signed, fee_paid
EVENT_VOLUNTEER	Events	Vol ID, event FK, volunteer FK, type, dbs_verified, checked_in
EVENT_REPORT	Events	Report ID, event FK, attendance, cost, incidents, feedback

MESSAGE	Comms	Message ID, sender FK, type, audience, channels, body, sent_at, receipts
NOTIFICATION_RULE	Comms	Rule ID, tenant FK, trigger, target_roles, channels, template, enabled
AUDIT_LOG	System	Log ID, tenant FK, actor FK, action, entity, old/new values, timestamp

3.2 The 23 Business Objects

Business Objects assemble data from multiple entities, add computed properties, and enforce business rules. The presentation layer interacts exclusively with BOs — never with raw entities.

ID	Name	Domain	Source Entities
BO-01	StudentProfile	Student	STUDENT, PARENT, EMERGENCY_CONTACT, STUDENT_HEALTH, APPLICATION, DOCUMENT
BO-02	EnrollmentCase	Student	APPLICATION, DOCUMENT, STUDENT, PARENT, INVOICE, PAYMENT
BO-03	AcademicCalendar	Academic	ACADEMIC_CALENDAR, EXAM, EVENT, LEAVE_REQUEST, FEE_STRUCTURE
BO-04	AcademicRecord	Academic	MARKS_RECORD, REPORT_CARD, ASSIGNMENT, STUDENT_ATTENDANCE, CURRICULUM
BO-05	ClassroomPlan	Academic	LESSON_PLAN, ASSIGNMENT, STUDENT_ATTENDANCE, CURRICULUM, SCHEDULE
BO-06	ExamPackage	Academic	EXAM, ACADEMIC_CALENDAR, MARKS_RECORD, SCHEDULE, STAFF
BO-07	AttendanceSheet	Attendance	STUDENT_ATTENDANCE, ATTENDANCE_ROSTER, STUDENT, PARENT, LEAVE_REQUEST
BO-08	LeaveCase	Attendance	LEAVE_REQUEST, STAFF_ATTENDANCE, STUDENT_ATTENDANCE, SCHEDULE
BO-09	StaffRoster	Attendance	STAFF_ATTENDANCE, SCHEDULE, STAFF, LEAVE_REQUEST
BO-10	FeeAccount	Finance	INVOICE, PAYMENT, DISCOUNT_WAIVER, FEE_STRUCTURE, STUDENT, PARENT
BO-11	PaymentTransaction	Finance	PAYMENT, INVOICE, PARENT, RECONCILIATION
BO-12	FinancialStatement	Finance	RECONCILIATION, INVOICE, PAYMENT, PAYROLL_RUN, VENDOR_INVOICE, DISCOUNT_WAIVER
BO-13	StaffMember	HR	STAFF, CONTRACT, CPD_RECORD, SCHEDULE, LEAVE_REQUEST, APPRAISAL
BO-14	RecruitmentCase	HR	STAFF (pre-hire), CONTRACT, CPD_RECORD, SCHEDULE

BO-15	PayrollRun	HR	PAYROLL_RUN, STAFF, STAFF_ATTENDANCE, LEAVE_REQUEST, SCHEDULE
BO-16	PerformanceRecord	HR	APPRAISAL, CPD_RECORD, STAFF, STAFF_ATTENDANCE, LEAVE_REQUEST
BO-17	HealthRecord	Health	STUDENT_HEALTH, HEALTH_OBSERVATION, MEDICATION_LOG, INCIDENT, EMERGENCY_CONTACT
BO-18	SafetyCompliance	Health	SAFETY_DRILL, INCIDENT, STUDENT_HEALTH, STAFF, AUDIT_LOG
BO-19	ProcurementOrder	Operations	REQUISITION, PURCHASE_ORDER, DELIVERY_RECORD, VENDOR_INVOICE, VENDOR, INVENTORY
BO-20	VendorAccount	Operations	VENDOR, PURCHASE_ORDER, VENDOR_INVOICE, DELIVERY_RECORD, PAYMENT
BO-21	SchoolEvent	Operations	EVENT, EVENT_REGISTRATION, EVENT_VOLUNTEER, EVENT_REPORT, INVOICE, INCIDENT, ACADEMIC_CALENDAR
BO-22	CommunicationBundle	Comms	MESSAGE, NOTIFICATION_RULE, PARENT, STAFF, STUDENT, AUDIT_LOG
BO-23	AnalyticsDashboard	Analytics	All 22 BOs (read-only aggregation) + AUDIT_LOG

4. Entity Relationship Summary

The full interactive ER diagram is available in the companion HTML file (purple_cubby_erd.html). The key relationship patterns are documented below by domain cluster.

4.1 Key Relationship Patterns

```
Student cluster: TENANT ||--o{ STUDENT, STUDENT ||--o{ APPLICATION, STUDENT ||--o{ PARENT,
STUDENT ||--|| STUDENT_HEALTH, APPLICATION ||--o{ DOCUMENT
Academic cluster: CURRICULUM ||--o{ LESSON_PLAN, ACADEMIC_CALENDAR ||--o{ EXAM, EXAM ||--o{
MARKS_RECORD, MARKS_RECORD ||--o{ REPORT_CARD
Attendance cluster: STUDENT ||--o{ STUDENT_ATTENDANCE, STAFF ||--o{ STAFF_ATTENDANCE, STUDENT
||--o{ LEAVE_REQUEST, ATTENDANCE_ROSTER ||--o{ STUDENT_ATTENDANCE
Billing cluster: FEE_STRUCTURE ||--o{ INVOICE, INVOICE ||--o{ PAYMENT, INVOICE ||--o{
DISCOUNT_WAIVER, PAYMENT }o--|| RECONCILIATION
HR cluster: STAFF ||--|| CONTRACT, STAFF ||--o{ PAYROLL_RUN, STAFF ||--o{ APPRAISAL, STAFF
||--o{ CPD_RECORD, STAFF ||--o{ SCHEDULE
Vendor cluster: VENDOR ||--o{ PURCHASE_ORDER, REQUISITION ||--|| PURCHASE_ORDER,
PURCHASE_ORDER ||--o{ DELIVERY_RECORD, PURCHASE_ORDER ||--o{ VENDOR_INVOICE
Events cluster: EVENT ||--o{ EVENT_REGISTRATION, EVENT ||--o{ EVENT_VOLUNTEER, EVENT ||--||
EVENT_REPORT, STUDENT }o--o{ EVENT_REGISTRATION
Comms cluster: MESSAGE ||--o{ NOTIFICATION_RULE (triggered by), AUDIT_LOG references all
entities
```

4.2 Cross-Domain Relationships

From	To	Relationship
EnrollmentCase (BO-02)	FeeAccount (BO-10)	Enrollment confirmation triggers first invoice generation
AcademicCalendar (BO-03)	All 10 Processes	Single source of truth for all date validation
AttendanceSheet (BO-07)	PayrollRun (BO-15)	Staff absence data feeds payroll deductions
AttendanceSheet (BO-07)	AcademicRecord (BO-04)	Attendance eligibility gates report card publication
SchoolEvent (BO-21)	FeeAccount (BO-10)	Event registration auto-generates invoice line item
ProcurementOrder (BO-19)	FinancialStatement (BO-12)	Vendor spend feeds expense dashboards
SafetyCompliance (BO-18)	AnalyticsDashboard (BO-23)	Drill and incident data feeds compliance KPIs

CommunicationBundle (BO-22)	All BOs	Triggered by all 10 process modules for notifications
--	----------------	---

P01 — Enrollment & Admissions

WHAT THIS PROCESS DOES

Manages the journey from a student application through document verification, admission decision, payment/seat confirmation, and creation of an active student profile. It ensures required documents, emergency-contact information, health records, and payment are in place before enrollment becomes active.

Simple process flow



BUSINESS OBJECTS

- BO-02 EnrollmentCase
- BO-01 StudentProfile

ENTITIES

- APPLICATION
- DOCUMENT
- STUDENT
- PARENT
- INVOICE
- PAYMENT

BUSINESS RULES

- Application cannot advance to Offered status without all required documents verified and marked as such by Admin
- Seat confirmation requires a payment receipt — enrollment is not active until payment is confirmed
- Waitlist position auto-recalculates when any confirmed applicant withdraws
- Student profile and health record must be created before the first scheduled class day
- At least one emergency contact must be recorded before enrollment is marked Active
- Admission decision (Approved/Waitlisted/Rejected) is immutable after notification is sent — only Owner can override

API Surface

POST /enrollments	POST /enrollments/{id}/documents
GET /enrollments/{id}/status	GET /enrollments/pipeline
PUT /enrollments/{id}/decision	GET /students/{id}/profile

P02 — Billing & Fee Management

WHAT THIS PROCESS DOES Manages the complete student-fee lifecycle: configure fees, generate invoices, apply approved discounts/waivers, record payments, calculate late fees, process refunds, and reconcile the financial period. Vendor invoices participate in the same reconciliation cycle.

Simple process flow



BUSINESS OBJECTS

- BO-10 FeeAccount
- BO-11 PaymentTransaction
- BO-12 FinancialStatement

ENTITIES

- FEE_STRUCTURE
- INVOICE
- PAYMENT
- DISCOUNT_WAIVER
- RECONCILIATION

BUSINESS RULES

- Invoice cannot be marked Paid if outstanding balance is greater than zero
- Late fee applies automatically after due date plus configured grace period — no manual trigger required
- Discount or waiver requires Principal approval; high-value waivers require Owner approval
- All discount approvals are logged with approver ID, timestamp, and reason — immutable
- Payment refunds above the configured threshold require Owner approval before processing
- Period reconciliation can only be finalised after all discrepancies are resolved or acknowledged
- Vendor invoices are processed through the same reconciliation cycle as student fee income

API Surface

GET /students/{id}/fee-account	POST /payments/{id}/refund
POST /invoices	POST /discounts
POST /invoices/bulk-generate	GET /financials/reconciliation/{period}
POST /payments	GET /financials/statement/{period}
GET /payments/{id}/receipt	

P03 — Attendance Management

WHAT THIS PROCESS DOES Records and controls student and staff attendance, leave, and substitutions. It validates when attendance can be marked, locks records for audit integrity, alerts parents or school leaders when attendance issues occur, and provides roster/reporting views.

Simple process flow



BUSINESS OBJECTS

- BO-07 AttendanceSheet
- BO-08 LeaveCase
- BO-09 StaffRoster

ENTITIES

- STUDENT_ATTENDANCE
- STAFF_ATTENDANCE
- LEAVE_REQUEST
- ATTENDANCE_ROSTER

BUSINESS RULES

- Attendance can only be marked within the configured attendance window for that class
- Attendance records are locked for editing 24 hours after the session date for audit integrity
- Absent without an approved leave request triggers an automated parent notification within the configured delay
- Staff absence without leave approval triggers a class-without-cover alert to Vice Principal immediately
- Monthly attendance below the configured threshold flags the student for counsellor review
- Geofence validation required for mobile check-in for staff attendance
- Leave clash with exam or blackout dates requires explicit Principal override

API Surface

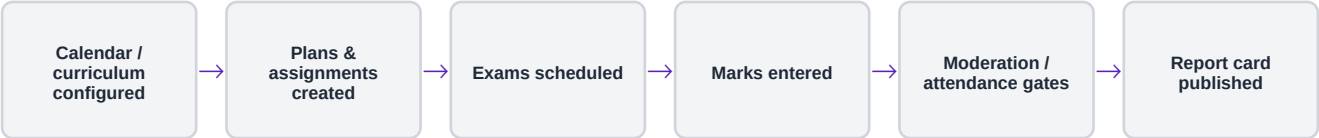
POST /attendance/mark
GET /classes/{id}/attendance/{date}
GET /students/{id}/attendance-summary
POST /leave-requests

PUT /leave-requests/{id}/approve
GET /staff/roster/{date}
POST /substitutions
GET /attendance/reports/{type}

P04 — Academic Management

WHAT THIS PROCESS DOES Coordinates curriculum, lesson planning, assignments, exams, marks, and report cards around the Academic Calendar. The calendar acts as the shared date-validation source, while moderation, attendance, and publication gates control when academic results can be released.

Simple process flow



BUSINESS OBJECTS

- BO-03 AcademicCalendar
- BO-04 AcademicRecord
- BO-05 ClassroomPlan
- BO-06 ExamPackage

ENTITIES

- CURRICULUM
- ACADEMIC_CALENDAR
- LESSON_PLAN
- ASSIGNMENT
- EXAM
- MARKS_RECORD
- REPORT_CARD

BUSINESS RULES

- Lesson plan must be mapped to at least one syllabus topic from the current curriculum version
- Assignment due date cannot fall on a calendar holiday or blackout date
- Exam cannot be scheduled during a holiday; two exams for the same class cannot overlap
- Marks cannot be published before Principal or VP moderation sign-off
- Report card release is gated by the published date configured in Academic Calendar — cannot be released early
- Student attendance below threshold blocks report card from being published to parent portal
- Academic Calendar is the single source of truth for all date validation across all 10 process modules
- Marks records are locked after the configured deadline; retroactive changes require Principal approval and audit note

API Surface

GET /calendar/current
POST /lesson-plans
GET /classes/{id}/plan/{week}
POST /assignments
POST /exams

POST /exams/{id}/marks
GET /students/{id}/academic-record/{term}
POST /academic-records/{id}/publish
GET /exams/{id}/hall-tickets

P05 — Staff & HR Management

WHAT THIS PROCESS DOES

Manages recruitment and staff lifecycle activities including vacancy progression, onboarding, contracts, scheduling, attendance inputs, payroll, appraisals, and CPD. It enforces compliance and approval gates before staff are scheduled or payroll is released.

Simple process flow



BUSINESS OBJECTS

- BO-13 StaffMember
- BO-14 RecruitmentCase
- BO-15 PayrollRun
- BO-16 PerformanceRecord

ENTITIES

- STAFF
- CONTRACT
- PAYROLL_RUN
- APPRAISAL
- CPD_RECORD
- SCHEDULE

BUSINESS RULES

- A staff member cannot be scheduled to teach if their DBS/background check is expired
- Contract renewal alert is sent to Admin and Principal 60 days before expiry
- Onboarding checklist must be 100% complete before the new joiner is assigned to their first class
- Employment contract must be e-signed before the RecruitmentCase graduates to a StaffMember BO
- Payroll cannot be disbursed without Owner approval of the payroll run
- Attendance data must be locked before payroll computation begins for that period
- Both self-assessment and manager assessment are required before an appraisal record can be finalised
- CPD hours below annual minimum blocks an Exceeds Expectations outcome in appraisal
- Substitute staff cannot be scheduled on their own approved leave day

API Surface

```
POST /recruitment/vacancies
GET /recruitment/{id}
PUT /recruitment/{id}/stage
POST /staff
GET /staff/{id}
GET /staff/{id}/compliance-status
```

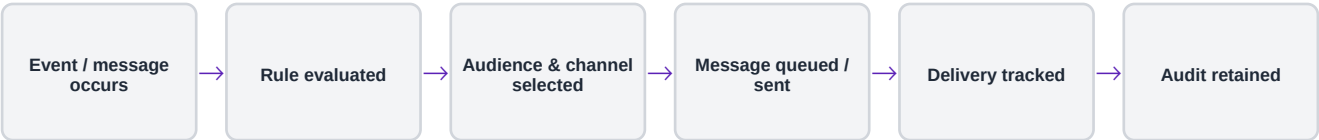
```
POST /payroll/compute/{period}
PUT /payroll/{id}/approve
GET /staff/{id}/payslip/{period}
POST /appraisals
GET /staff/{id}/performance-history
```

P06 — Communication & Notifications

WHAT THIS PROCESS DOES

Provides a central communication engine for user messages and automated notifications. It evaluates notification rules, routes messages through configured channels, records delivery/audit history, and supports controlled broadcasts and emergency communications.

Simple process flow



BUSINESS OBJECTS

- BO-22 CommunicationBundle
- BO-22b NotificationRuleSet

ENTITIES

- MESSAGE
- NOTIFICATION_RULE
- AUDIT_LOG

BUSINESS RULES

- Emergency broadcast messages bypass all individual user notification opt-out preferences
- Bulk messages to more than one class or role group require Admin role or above
- All messages — including automated system notifications — are logged immutably in the audit trail
- Notification rule changes are versioned; rollback to previous version is available for 30 days
- Template changes require a preview-and-approve step before the rule goes live
- Parent-teacher messages must be retained for a minimum of 12 months for safeguarding purposes
- Out-of-hours messages are queued and delivered at the start of the next available window unless flagged as urgent

API Surface

POST /messages
GET /messages/{id}/delivery-status
POST /messages/broadcast
GET /notification-rules

POST /notification-rules
PUT /notification-rules/{id}
GET /notification-rules/{id}/history
POST /messages/emergency

P07 — Health & Safety Management

WHAT THIS PROCESS DOES Maintains student health information and day-to-day observations, medication records, incidents, and safety drills. It applies consent and escalation controls, preserves immutable safety records, and produces compliance reporting.

Simple process flow



BUSINESS OBJECTS

- BO-17 HealthRecord
- BO-18 SafetyCompliance

ENTITIES

- STUDENT_HEALTH
- HEALTH_OBSERVATION
- MEDICATION_LOG
- INCIDENT
- SAFETY_DRILL

BUSINESS RULES

- Medication administration requires a signed parent consent form on record before the first dose
- Any incident must be acknowledged and categorised by Admin within 1 hour of logging
- Serious incidents (injury, allergic reaction) are automatically escalated to Principal and trigger parent notification
- Safety drill records are immutable after sign-off — corrections require a new superseding record
- Compliance report is auto-generated at month-end and cannot be suppressed by any role
- Emergency contact records must be verified before a student attends any off-site event
- Health profile updates by parents are logged with timestamp and version for safeguarding records

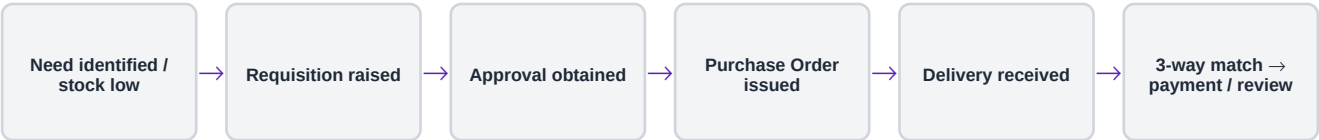
API Surface

GET /students/{id}/health-record	GET /incidents/{id}
POST /health/observations	POST /safety/drills
POST /health/medication-log	GET /safety/compliance-report/{period}
POST /incidents	GET /safety/compliance-dashboard

P08 — Vendor & Procurement

WHAT THIS PROCESS DOES Controls purchasing from request through approval, purchase order, delivery, invoice matching, and payment, while also managing vendor accounts and inventory thresholds. The process blocks payment when the PO, delivery, and invoice do not match.

Simple process flow



BUSINESS OBJECTS

- BO-19 ProcurementOrder
- BO-20 VendorAccount

ENTITIES

- VENDOR
- REQUISITION
- PURCHASE_ORDER
- DELIVERY_RECORD
- VENDOR_INVOICE
- INVENTORY

BUSINESS RULES

- A Purchase Order cannot be raised without an approved requisition on record
- Vendor invoice 3-way match failure (PO / delivery / invoice) automatically blocks payment and flags for review
- Payment cannot exceed the PO value without a new approval at the original approval tier
- Suspended vendors cannot receive new POs until suspension is lifted by Owner
- Preferred vendor status requires a minimum 12-month track record and performance score above threshold
- Inventory items with stock below minimum threshold automatically generate a requisition draft for Admin review
- High-value vendor contracts require Board approval before the vendor is added to the Preferred Vendor List

API Surface

POST /vendors	GET /procurement/orders/{id}
GET /vendors/{id}	POST /procurement/orders/{id}/receive
GET /vendors/{id}/account	POST /procurement/invoices/{id}/approve
POST /procurement/requisitions	GET /inventory
PUT /procurement/requisitions/{id}/approve	POST /inventory/adjust
POST /procurement/orders	

P09 — Events & Extra-Curricular

WHAT THIS PROCESS DOES

Plans and runs school events from creation and approval through parent registration, permissions, volunteer coordination, fees, attendance, and post-event reporting. It uses the Academic Calendar and safety checks to control when and how events can proceed.

Simple process flow



BUSINESS OBJECTS

- BO-21 SchoolEvent

ENTITIES

- EVENT
- EVENT_REGISTRATION
- EVENT_VOLUNTEER
- EVENT_REPORT
- ACADEMIC_CALENDAR

BUSINESS RULES

- Event cannot be published to parents before Principal approval
- Permission slip deadline must be set at least 48 hours before the event date
- Event fees are automatically added to the next student invoice cycle — no manual step required
- Off-site events require all registered students to have a verified emergency contact on record
- Volunteer DBS/background check status is validated automatically against the staff or parent profile
- Post-event report must be completed within 5 working days; Principal is notified if overdue
- Events cannot be scheduled on Academic Calendar blackout dates without Owner override

API Surface

POST /events
GET /events/{id}
PUT /events/{id}/approve
POST /events/{id}/register

POST /events/{id}/volunteers
GET /events/{id}/dashboard
POST /events/{id}/report
GET /events/calendar

P10 — Reports & Analytics

WHAT THIS PROCESS DOES Aggregates trusted data from the platform's Business Objects into role-specific dashboards, KPI snapshots, custom reports, scheduled deliveries, and compliance reporting. It is read-only at the BO layer and preserves governance controls for exports and cross-tenant views.

Simple process flow



BUSINESS OBJECTS

- BO-23 AnalyticsDashboard
- BO-12 FinancialStatement
- BO-04 AcademicRecord
- BO-07 AttendanceSheet

ENTITIES

- All 46 entities via BO aggregation — no direct entity reads

BUSINESS RULES

- KPI dashboard snapshots are pre-computed on a schedule — not generated on demand — to protect system performance
- Custom report definitions are saved as versioned templates owned by their creator
- Scheduled report delivery uses CommunicationBundle (BO-22) — all delivery is tracked and logged
- Cross-tenant comparative data is visible to Owner role only — never exposed to school-level roles
- Compliance reports are auto-delivered to configured regulatory contacts and cannot be suppressed
- AnalyticsDashboard BO reads only from other BOs — never directly from base entities
- Data export includes an audit timestamp and the requesting user ID for data governance

API Surface

GET /analytics/dashboard	GET /reports/templates
GET /analytics/kpis/{role}	GET /analytics/enrollment-funnel
POST /reports/custom	GET /analytics/financial/{period}
GET /reports/{id}	GET /analytics/compliance/{period}
POST /reports/{id}/schedule	

6. API Architecture

6.1 API Design Principles

- RESTful resource-based URLs — nouns, not verbs
- All endpoints require JWT bearer token in Authorization header
- Tenant scoping is automatic via JWT claims — no `tenant_id` in URL
- Standard response envelope: { data, meta, errors }
- Pagination: cursor-based for large collections, page-based for reports
- Versioning: `/api/v1/` prefix — breaking changes increment the major version
- Rate limiting: 1000 req/min per tenant for Standard, 5000 for Enterprise
- All mutation endpoints return the updated BO, not just a success flag

6.2 Standard Response Envelope

```
GET /api/v1/students/{id}/fee-account
{
  "data": {
    "bo": "FeeAccount",
    "student_id": "uuid",
    "outstanding_balance": 1200.00,
    "next_due_date": "2025-09-01",
    "invoices": [ ... ],
    "computed_at": "2025-08-25T10:00:00Z"
  },
  "meta": { "tenant_id": "uuid", "role": "parent", "version": "v1" },
  "errors": null
}
```

6.3 Error Response

```
{
  "data": null,
  "meta": { ... },
  "errors": [
    { "code": "RULE_VIOLATION", "field": "due_date",
      "message": "Due date falls on a calendar holiday" }
  ]
}
```

6.4 API Endpoint Catalogue

P	BO	Endpoints
P01	BO-02 EnrollmentCase	POST /enrollments · GET /enrollments/{id}/status · PUT /enrollments/{id}/decision · POST /enrollments/{id}/documents · GET /enrollments/pipeline · GET /students/{id}/profile
P02	BO-10 FeeAccount	GET /students/{id}/fee-account · POST /invoices · POST /invoices/bulk-generate · POST /payments · GET /payments/{id}/receipt · POST /payments/{id}/refund · POST /discounts · GET /financials/reconciliation/{period} · GET /financials/statement/{period}
P03	BO-07 AttendanceSheet	POST /attendance/mark · GET /classes/{id}/attendance/{date} · GET /students/{id}/attendance-summary · POST /leave-requests · PUT /leave-requests/{id}/approve · GET /staff/roster/{date} · POST /substitutions · GET /attendance/reports/{type}
P04	BO-03 AcademicCalendar	GET /calendar/current · POST /lesson-plans · GET /classes/{id}/plan/{week} · POST /assignments · POST /exams · POST /exams/{id}/marks · GET /students/{id}/academic-record/{term} · POST /academic-records/{id}/publish · GET /exams/{id}/hall-tickets
P05	BO-13 StaffMember	POST /recruitment/vacancies · GET /recruitment/{id} · PUT /recruitment/{id}/stage · POST /staff · GET /staff/{id} · GET /staff/{id}/compliance-status · POST /payroll/compute/{period} · PUT /payroll/{id}/approve · GET /staff/{id}/payslip/{period} · POST /appraisals · GET /staff/{id}/performance-history
P06	BO-22 CommunicationBundle	POST /messages · GET /messages/{id}/delivery-status · POST /messages/broadcast · GET /notification-rules · POST /notification-rules · PUT /notification-rules/{id} · GET /notification-rules/{id}/history · POST /messages/emergency
P07	BO-17 HealthRecord	GET /students/{id}/health-record · POST /health/observations · POST /health/medication-log · POST /incidents · GET /incidents/{id} · POST /safety/drills · GET /safety/compliance-report/{period} · GET /safety/compliance-dashboard
P08	BO-19 ProcurementOrder	POST /vendors · GET /vendors/{id} · GET /vendors/{id}/account · POST /procurement/requisitions · PUT /procurement/requisitions/{id}/approve · POST /procurement/orders · GET /procurement/orders/{id} · POST /procurement/orders/{id}/receive · POST /procurement/invoices/{id}/approve · GET /inventory · POST /inventory/adjust
P09	BO-21 SchoolEvent	POST /events · GET /events/{id} · PUT /events/{id}/approve · POST /events/{id}/register · POST /events/{id}/volunteers · GET /events/{id}/dashboard · POST /events/{id}/report · GET /events/calendar
P10	BO-23 AnalyticsDashboard	GET /analytics/dashboard · GET /analytics/kpis/{role} · POST /reports/custom · GET /reports/{id} · POST /reports/{id}/schedule · GET /reports/templates · GET /analytics/enrollment-funnel · GET /analytics/financial/{period} · GET /analytics/compliance/{period}

6.5 Webhook Events

Enterprise tenants can subscribe to system events via webhooks. Each event payload includes the BO name, event type, tenant_id, actor_id, and the changed BO snapshot.

```
student.enrolled  
invoice.generated  
payment.completed  
payment.overdue  
attendance.marked.absent  
incident.created  
incident.escalated  
exam.published  
report_card.published  
staff.contract.expiring  
procurement.po.approved  
vendor.invoice.matched  
event.registration.closed  
compliance.report.generated
```

7. Development & QA Approach — 5 Sprints

Sprint 1 · Foundation & Tenant Core Weeks 1–2

Multi-tenant infrastructure, authentication, role-based access, and core entity layer operational.

Development Tasks	QA & Testing
- Multi-tenant schema with row-level security (tenant_id on all tables) - Auth service: JWT, refresh tokens, 2FA for Owner/Admin/Board roles - RBAC engine: role definitions, permission matrix, portal routing - Base entity DDL: TENANT, STUDENT, PARENT, EMERGENCY_CONTACT, DOCUMENT, APPLICATION - STUDENT domain BOs: StudentProfile (BO-01), EnrollmentCase (BO-02) - Enrollment APIs: P01 full API surface - Parent portal skeleton + Enrollment flow UI - Admin console: application queue, document verification, decision workflow - Audit logging infrastructure (immutable, all mutations captured)	- Unit tests: tenant isolation — cross-tenant data leakage tests must all fail - Unit tests: RBAC — each role cannot access endpoints outside their permission set - Integration tests: full enrollment lifecycle P01 (inquiry → active student) - Security: SQL injection, auth bypass, tenant spoofing penetration tests - Performance: enrollment endpoint <200ms p95 under 50 concurrent users

Sprint 2 · Academic & Attendance Core Weeks 3–4

Academic Calendar as system-wide configuration, full academic management, and attendance tracking live.

Development Tasks	QA & Testing
- ACADEMIC_CALENDAR entity + AcademicCalendar BO (BO-03) — cross-module validation hook - Academic entities: CURRICULUM, LESSON_PLAN, ASSIGNMENT, EXAM, MARKS_RECORD, REPORT_CARD - Academic BOs: AcademicRecord (BO-04), ClassroomPlan (BO-05), ExamPackage (BO-06) - Attendance entities: STUDENT_ATTENDANCE, STAFF_ATTENDANCE, LEAVE_REQUEST, ATTENDANCE_ROSTER - Attendance BOs: AttendanceSheet (BO-07), LeaveCase (BO-08), StaffRoster (BO-09) - P04 Academic Management full API surface - P03 Attendance Management full API surface	- Unit tests: AcademicCalendar validation rules (no exam on holiday, leave blackout) - Unit tests: attendance window enforcement, 24-hour lock - Integration tests: marks → report card pipeline including moderation gate - Integration tests: absent student → parent notification chain - E2E: Teacher marks attendance → parent receives SMS within configured window - Data integrity: marks locked after deadline cannot be edited via API

- Teacher portal: lesson plans, assignments, marks entry, attendance marking
- Care Giver portal: session attendance, arrival/departure times
- Parent portal: child academic view, attendance view, leave request
- QR code attendance integration

Sprint 3 · Finance & HR Weeks 5–6

Full billing lifecycle, payroll computation, HR management, and recruitment pipeline operational.

Development Tasks	QA & Testing
• Billing entities: FEE_STRUCTURE, INVOICE, PAYMENT, DISCOUNT_WAIVER, RECONCILIATION • Finance BOs: FeeAccount (BO-10), PaymentTransaction (BO-11), FinancialStatement (BO-12) • Payment gateway integration (card, UPI, bank transfer) • Auto-invoice generation job, late fee computation, overdue reminder scheduler • HR entities: STAFF, CONTRACT, PAYROLL_RUN, APPRAISAL, CPD_RECORD, SCHEDULE • HR BOs: StaffMember (BO-13), RecruitmentCase (BO-14), PayrollRun (BO-15), PerformanceRecord (BO-16) • P02 Billing full API surface + P05 HR full API surface • Admin console: fee management, invoice dashboard, payment recording, reconciliation • Owner portal: payroll approval, financial dashboards • Staff portal: payslip view, leave requests, profile management • Recruitment pipeline UI: vacancy → shortlist → interview → offer → onboarding	• Unit tests: late fee auto-application, discount approval gate, refund threshold • Unit tests: payroll computation across all deduction and allowance combinations • Integration tests: full billing cycle (fee structure → invoice → payment → reconciliation) • Integration tests: recruitment lifecycle (vacancy → StaffMember BO created) • Security: payment endpoint PCI-DSS compliance check • Performance: invoice bulk generation for 500 students <5 seconds • Regression: enrollment completion still triggers first invoice (P01→P02 integration)

Sprint 4 · Health, Vendor, Events & Comms Weeks 7–8

All remaining operational modules complete — health, procurement, events, and communication engine live.

Development Tasks	QA & Testing
• Health entities: STUDENT_HEALTH, HEALTH_OBSERVATION, MEDICATION_LOG, INCIDENT, SAFETY_DRILL	• Unit tests: 3-way match logic, vendor suspension blocking new POs • Unit tests: medication consent gate, incident escalation rules

• Health BOs: HealthRecord (BO-17), SafetyCompliance (BO-18) • Vendor entities: VENDOR, REQUISITION, PURCHASE_ORDER, DELIVERY_RECORD, VENDOR_INVOICE, INVENTORY • Vendor BOs: ProcurementOrder (BO-19), VendorAccount (BO-20) • Event entities: EVENT, EVENT_REGISTRATION, EVENT_VOLUNTEER, EVENT_REPORT • Event BO: SchoolEvent (BO-21) • Comms entities: MESSAGE, NOTIFICATION_RULE • Comms BOs: CommunicationBundle (BO-22), NotificationRuleSet • P07 Health, P08 Vendor, P09 Events, P06 Comms full API surfaces • Notification engine: trigger evaluation, channel routing, SMS/email/push dispatch • Vendor portal: invoice submission, PO tracker, payment status • Care Giver portal: medication log, health observations, incident reporting • Event management UI: planning, registration, volunteer coordination, post-event report • Emergency broadcast feature with delivery confirmation	• Unit tests: notification rule evaluation for all 12 trigger event types • Integration tests: full procurement lifecycle (requisition → PO → delivery → invoice → payment) • Integration tests: event fee auto-invoice generation • E2E: incident logged by teacher → parent notified → Admin acknowledged within 1 hour • E2E: emergency broadcast → all parent SMS delivered → delivery report generated • Load test: notification engine under 1000 concurrent triggers
--	--

Sprint 5 · Analytics, Hardening & Launch Weeks 9–10

Analytics layer complete, all cross-module integrations verified, performance hardened, and system production-ready.

Development Tasks	QA & Testing
• AnalyticsDashboard BO (BO-23): reads from all 22 BOs, never direct entity access • KPI snapshot scheduler: pre-computed dashboards per role, configurable refresh interval • Custom report builder: field selector, filter, grouping, export (PDF/Excel/CSV) • Scheduled report delivery via CommunicationBundle • P10 Reports & Analytics full API surface • Owner portal: multi-school comparative dashboards, network-wide KPIs • Board/Trustee portal: governance and compliance report pack • Mobile optimisation: PWA for Teacher and Care Giver portals	• Full regression suite across all 10 process modules • Cross-module integration tests: P01→P02, P03→P05, P09→P02, P04→P03 integration chains • Performance: dashboard load <1 second for 95th percentile, analytics queries <3 seconds • Load test: 200 concurrent users across all portals simultaneously • Security: full penetration test — OWASP Top 10, tenant isolation, privilege escalation • Accessibility audit: WCAG 2.1 AA for all portal UIs • Data governance: export audit trail, GDPR erasure request flow • UAT: each of the 10 user roles sign off their portal with real school data

- | | |
|---|---|
| <ul style="list-style-type: none">• Webhook framework: allow tenants to subscribe to system events• Data export and GDPR/data protection compliance tooling• Process configuration UI: Admin interface to create/modify processes (see Section 8)• Final cross-module integration wiring and smoke test across all P01–P10 paths | <ul style="list-style-type: none">• DR drill: backup restore test, RTO/RPO validation |
|---|---|

8. Creating & Modifying Processes

The Purple Cubby is designed for process extensibility. New process modules can be added and existing ones modified by following the governance framework below. No code changes are required for configuration-level modifications; structural changes (new entities or BOs) follow the structured engineering path.

8.1 Two Types of Change

Type	What it covers	Who can do it
Configuration change	Notification rules, fee components, attendance windows, grading scales, calendar dates, report templates	Admin (within their school) or Owner (network-wide) — no engineering required
Structural change	New process module, new entity, new BO, new API endpoint, new portal role	Engineering team following the process below — requires code review and deployment

8.2 Configuration-Level Modification (No Code)

The following can be changed by authorised users through the Admin Console without any engineering involvement:

Fee Structure

- Navigate to Admin Console → Billing → Fee Structures
- Select the grade and term to modify
- Add, edit, or remove fee components
- Configure discount rules and penalty schedules
- Set effective date — changes apply from that date forward, not retroactively
- Save and publish — all future invoices use the new structure

Notification Rules

- Navigate to Admin Console → Communications → Notification Rules
- Select the trigger event to modify (e.g. attendance.marked.absent)
- Edit target roles, channels, delay window, and message template
- Click Preview to test the rule against a sample event
- Click Publish — the previous version is automatically archived (rollback available for 30 days)

Academic Calendar

- Navigate to Admin Console → Academics → Calendar
- Select the academic year
- Add or edit term dates, holiday dates, exam weeks, and blackout periods
- All dependent modules (attendance, exams, leave, events) immediately use the updated calendar
- Cannot delete a date that has existing exams or events — must reschedule those first

8.3 Structural Change — Adding a New Process

Follow these steps to add a new process module (e.g. Library Management, Transport Tracking):

Step 1: Process Definition Document

- Author a Process Definition using the standard template (see 8.4)
- Specify: process name, description, steps, roles involved, business rules, and integration points
- Get sign-off from Owner/Principal stakeholder and Technical Lead
- Register the process in the Process Registry (Admin Console → System → Process Registry)

Step 2: Entity Design

- Identify new entities required — check if existing entities can serve the need first
- For each new entity: define all fields, data types, constraints, and FK relationships
- Author the entity in the ER diagram file and add to the 46-entity catalogue
- Submit a database migration script following the project migration standards
- All new entities must include: `tenant_id`, `created_at`, `updated_at`, `created_by`, `is_deleted` (soft delete)

Step 3: Business Object Design

- Define the new BO(s) required using the BO template (see 8.5)
- Specify: source entities, computed properties, business rules, state machine (if applicable)
- Map to which portals will consume this BO and with what access level
- Assign a BO-ID (next sequential number in the catalogue)
- Code review required: BO must be reviewed by at least one senior engineer

Step 4: API Design

- Define all new endpoints following the RESTful conventions in Section 6
- Add endpoints to the OpenAPI specification file (`openapi.yaml`)
- Ensure each endpoint has: authentication, RBAC enforcement, input validation, error codes
- Add webhook event definitions for any new system events the process generates

Step 5: Implementation

- Create a feature branch: `feature/process-{name}`

- Implement entities (migration), BO class, service layer, API handlers in that order
- Write unit tests for all business rules (minimum 80% branch coverage)
- Write integration tests for the full process happy path and key failure paths
- Update the process flows HTML reference document

Step 6: QA & Release

- Submit pull request — requires 2 approvals from senior engineers
- QA team runs: unit tests, integration tests, regression suite, security scan
- Performance test: new endpoints must meet the <200ms p95 SLA
- Update tenant subscription configuration if the new process is tier-gated
- Deploy to staging → UAT with at least one real school → production

8.4 Process Definition Template

Copy this template to create the Process Definition Document for any new process.

```
# Process Definition: [PROCESS NAME]
```

Overview

Process ID: P{nn}

Name: [Process Name]

Description: [One paragraph describing what this process covers]

Subscription Tier: [Basic / Standard / Enterprise]

Steps

1. [Step name] — [Step description, roles involved, data touched]

2. ...

Business Objects

- BO-{id} {Name} — [how it is used in this process]

Entities

- {ENTITY_NAME} — [read / write / create]

Business Rules

- BR-{nn}: [Rule statement — clear, testable, unambiguous]

Role Access Matrix

Role	Access Level	Specific Capabilities
Admin	Full	...

---|---|---

| Admin | Full | ... |

API Endpoints

- METHOD /path - [description]

Integration Points

- [Which other processes does this trigger or depend on]

Notifications

- [Trigger event] → [Audience] → [Channel] → [Template name]

Approval

Product Owner: _____ Date: _____

Technical Lead: _____ Date: _____

8.5 Business Object Template

Business Object: {BO-ID} {Name}

Domain: [Student / Academic / Attendance / Finance / HR / Health / Operations / Comms / Analytics]

Process(es): [P01, P04, ...]

Source Entities

- {ENTITY_NAME} - [fields used]

Computed Properties

- {property_name}: {type} - [description of computation]

Business Rules

- [Rule statement - must be enforceable in code]

State Machine (if applicable)

States: [State A] → [State B] → [State C]

Transitions:

[State A] → [State B]: trigger, guard condition, action

API Surface

- GET /resource/{id} - returns this BO
 - POST /resource - creates and returns this BO
 - PATCH /resource/{id} - updates and returns this BO

Portal Access

| Portal | Access | Notes |

```
| --- | --- | --- |  
| Admin | Full | ... |
```

```
## Dependencies  
- Reads from: [Other BOs this BO reads]  
- Read by: [BOs / portals that consume this BO]
```

8.6 Modifying an Existing Process

Adding a step to an existing process

- Open the Process Definition Document for the process
- Add the new step with: description, roles, entities touched, business rules, and API changes
- Assess impact: does the new step require a new entity or BO? If yes, follow the structural path
- If only configuration: update notification rules and calendar hooks as needed
- Update the process flows reference document and redeploy
- Notify affected portal users with a change summary via the Communications module

Modifying a business rule

- Locate the rule in the BO definition (each rule has a BR-{nn} identifier)
- Document the old and new rule in the change log
- Assess data impact: does changing the rule affect existing records? If yes, plan a data migration
- Update the BO class and its unit tests
- Submit for code review — business rule changes always require 2 approvals
- Announce the rule change to affected Admin users before deployment

Deprecating a process

- Mark the process as Deprecated in the Process Registry — do not delete
- Set a deprecation date: minimum 90 days notice to active tenants
- Disable the process for new tenants immediately
- Migrate existing tenant data to the replacement process or archive tables
- Remove API endpoints after the deprecation date (return 410 Gone, not 404)
- Archive the Process Definition Document — it must be retained for 7 years